Reflective Journal

Module 10 Lab: Unsupervised Learning
Group: FA26_ML_6171_16347 5 | Course: ITAI 1371

Group Members and Assigned Sections

Team Member	Section	Notebook Part
Jamie Lopez	Section 1	Part 1: Supervised vs. Unsupervised Learning, data setup and exploration
Kerry Walker	Section 2	Task 1: Elbow Method for the optimal number of clusters
Angel Martinez	Section 3	Task 2: K-Means clustering and visualization
Kristina Akpan	Section 4	Task 3: PCA on the Iris dataset
Thomas Jaymon	Section 5	Extension: PCA + K-Means combined workflow, Knowledge Check, and final notebook review

Jamie Lopez

Section 1: Part 1, Supervised vs. Unsupervised Learning, data setup and exploration

Honestly, I signed up for Section 1 because I thought it would be the easy one. Import some libraries, make some fake data, draw a scatter plot, done. That is not quite how it went.

The first surprise came before I wrote any code. The starter notebook had lost all of its line breaks, so each code cell was one long jammed line that would not run. I had to rebuild every cell by hand, figuring out where one statement ended and the next began. It was tedious, but it made me slow down and read each line instead of just hitting run. By the time I finished, I understood the setup better than I would have otherwise. I also put all the shared imports at the top, including numpy and seaborn, so my teammates' sections would not break later.

The second surprise was a single variable: y. When make_blobs creates the customer data, it hands back two things, the points in X and the true group of each point in y. My instinct from every earlier lab was to use y. In the Titanic labs, the answer column was the whole point. This time I had to deliberately set it aside. I even wrote a comment in the code saying we only keep y so we can check our work at the end. Typing that comment was the moment unsupervised learning made sense to me. It is like taking a test where nobody ever gives you an answer key, so you have to figure out the patterns on your own.

Then I plotted the data, and I had a funny reaction. I could see four groups right away. Why would anyone need an algorithm for this? It reminded me of the closet example from the slides, where you sort your clothes into groups without anyone telling you how. With two features, I can sort the closet myself. But imagine a closet with fifty kinds of measurements on every item. Nobody could see the groups then. That is where clustering actually earns its place. Our synthetic data is like

a practice run where we can see the answer, so we know the tool works before we trust it on data we cannot see.

I also added a quick summary of the data: 300 customers, 2 features, with means around negative 3.40 and 2.85 and standard deviations of 5.25 and 6.21. Those numbers looked boring at first. Then I remembered slide 27, which says to always standardize before K-Means. K-Means measures distance, so if one feature were in dollars and the other in single digits, the dollars would decide everything. Our two features were on similar scales, so we were fine. Later, Kristina showed in her PCA section what happens when the scales are not similar, and I liked seeing my little check connect to her bigger finding.

What I keep thinking about is how a real company would decide which features to use in the first place. Our data was made to separate cleanly. Real customers would come with spending, visit frequency, age, location, and a lot of noise. If you picked the wrong features, you could get groups that look neat on a chart and mean nothing in real life. I used to think the algorithm was the hard part. Now I suspect the hard part is everything that happens before you call it.

Reflective Journal

Module 10 Lab: Unsupervised Learning
Group: FA26_ML_6171_16347 5 | Course: ITAI 1371

Kerry Walker

Section 2: Task 1, Elbow Method for the optimal number of clusters

Assignment: determine the optimal number of clusters using the Elbow Method. Result: k = 4, confirmed by a second method. Below is how I got there and what I learned.

The code was straightforward. Loop k from 1 to 10. Fit K-Means each time. Store the inertia. Plot it. The real work was interpreting the curve without guessing.

I printed the inertia values instead of reading them off the chart. The numbers:

k = 1: 19,780.25. k = 2: 9,211.21. k = 3: 1,919.36. k = 4: 362.47. k = 5: 329.27. k = 10: 188.72.

From 3 to 4 clusters, inertia dropped by about 1,557. From 4 to 5, it dropped by about 33. That is a 47 times smaller improvement for the same added complexity. The elbow is not subtle. It is at 4.

Key insight: inertia always decreases as k increases. If every point is its own cluster, inertia equals zero. Zero looks perfect but describes nothing. This is the clustering version of overfitting, which we covered in earlier modules. The model memorizes individual points instead of finding structure. The goal is therefore not the minimum inertia. The goal is the point where extra clusters stop paying for themselves.

Problem: the elbow is still a visual judgment. I wanted a number. Slide 9 lists silhouette analysis, so I added it. Silhouette compares how close each point is to its own cluster against how close it is to the nearest other cluster. Range is negative 1 to 1. Higher is better.

Silhouette results: 0.603 at k = 2, 0.778 at k = 3, 0.834 at k = 4, 0.698 at k = 5, then continued decline. Peak at k = 4.

Two independent methods, same answer. Inertia measures how tight the clusters are. Silhouette measures tightness and separation together. Agreement between them moves k = 4 from a reasonable guess to a supported decision.

Two technical issues came up. First, silhouette needs at least 2 clusters, so that loop starts at 2, not 1. Second, I used argmax to find the best score, then had to map that index back to the correct k value. Off by one errors are easy to make there.

Limitation: our data was generated with four well separated blobs, which produced a sharp elbow. Real data will not always cooperate. A smooth curve with no clear bend is possible. In that case, I would weigh the silhouette score more heavily and consider practical constraints. For

example, if a business can only support three customer programs, three clusters may be the correct operational answer regardless of what the math prefers.

Takeaway: in supervised learning, accuracy gave us one number to optimize. In unsupervised learning, choosing k requires combining several pieces of evidence and then making a judgment call. That is less comfortable, but it is a more honest picture of how these decisions get made.

Reflective Journal

Module 10 Lab: Unsupervised Learning
Group: FA26_ML_6171_16347 5 | Course: ITAI 1371

Angel Martinez

Section 3: Task 2, K-Means clustering and visualization

Here is how I think about my section. Imagine you run an online store with 300 customers, and your boss asks, "Who are our customers, really?" You do not have labels. Nobody filled out a form saying "I am a bargain shopper." All you have is behavior. That is the problem K-Means solves, and slide 12 on customer segmentation is basically what my section was practicing.

My job was to take Kerry's answer of k = 4 and actually build the segments. I fit the model, pulled out the cluster labels and the centers, and plotted everything with a red X marking the center of each group. The model settled in just 3 iterations. I expected it to take longer, but when groups are this far apart, the centers find their spots almost right away.

Each cluster got exactly 75 customers. If this were a real store, I would look at each center and give the segment a name. One center sits at about 4.70 and 2.03, another way down at negative 6.84 on both features. In a real dataset those might be something like high spenders who rarely visit versus low spenders who visit constantly. The centroid is the typical customer for that group, which is a pretty useful thing to hand a marketing team.

Here is the part that tripped me up, though. Because our data was synthetic, we had the true groups hidden away. So I checked our clusters against them. The Adjusted Rand Index came out to 1.000, a perfect match. But when I printed the crosstab, the numbers did not line up. True group 1 was called cluster 2, and true group 2 was called cluster 1. My first thought was, did we break something? We did not. K-Means just hands out numbers as names. It knows which customers belong together, but it has no idea what to call them. That is exactly why I used the Adjusted Rand Index instead of plain accuracy. Accuracy would have marked a perfect grouping as half wrong just because the names were different.

That connected back to our Model Evaluation lab in a way I did not expect. Back then, we had confusion matrices because we had real answers. Here, we only had answers because we made the data ourselves. In the real world, a business would never get that answer key. Someone would have to look at the centers and decide what each group means. So the algorithm does the sorting, and people still have to do the thinking.

I also could not stop thinking about slide 15, the limitations. K-Means assumes groups are round and roughly the same size. Our data was built that way, so of course it worked perfectly. But what if one group of customers is huge and another is tiny? What if a group is shaped like a long

stretched line? I would bet K-Means struggles. That is something I would want to test by changing the spread of the blobs.

So what is my big takeaway? A perfect score on practice data does not mean a method is perfect. It means the method works when its assumptions hold. If I were actually presenting segments to a manager, I would want to know how messy the real data is before I promised anything.

Reflective Journal

Module 10 Lab: Unsupervised Learning
Group: FA26_ML_6171_16347 5 | Course: ITAI 1371

Kristina Akpan

Section 4: Task 3, PCA on the Iris dataset

I had PCA, which honestly was the part of this lab I was least sure about going in. Clustering made sense right away because it is just grouping stuff. PCA sounded more abstract, like rotating the data into new axes, and I did not really get why you would want to do that until I actually ran it.

The main task was to take Iris, which has 4 features, and squeeze it down to 2 so we could plot it. The result was that 2 components kept 97.77% of the variance. PC1 alone kept 92.46%, and PC2 added 5.31%. So we threw away half the columns and only lost about 2% of the information. That is what made it click for me. The 4 measurements are not really 4 separate pieces of information. They overlap a lot.

I wanted to know what PC1 actually was, so I added a cell to print the loadings. Petal length had the biggest weight on PC1 at 0.857, and sepal length and petal width were around 0.36. Basically PC1 is a size score. Bigger flowers score higher on it. That made the plot make sense too. Setosa sits on its own on the left because it has way smaller petals, and versicolor and virginica sit next to each other with a little overlap because their sizes are closer. I also changed the legend to show the species names instead of 0, 1, 2, because the numbers did not tell you anything when you looked at the chart. I also fixed a small issue in the starter code: it used sns.scatterplot but never imported seaborn, so I added the import in my cell.

The thing I am most glad I tried was the scaling check. Slide 24 says PCA is sensitive to scaling, and I wanted to see if that was true for Iris. When I standardized the features first, 2 components still kept 95.81%, but the split changed a lot: 72.96% and 22.85% instead of 92.46% and 5.31%. That showed me that petal length was kind of dominating the unscaled version just because it has the biggest range in centimeters. So the 97.77% number is real, but part of why PC1 looks so strong is the units. That was an actual surprise and it tied back to what Jamie checked in Section 1 about the blob features being on similar scales.

The hard part for me was keeping straight what PCA is and is not doing. It is not using the species labels at all. We only pass the labels in for coloring the plot. PCA is still unsupervised. It just finds the directions where the data spreads out the most. The fact that the species separate nicely on those directions is a bonus, not something PCA was trying to do.

I also added up the variance for all 4 components to see the full picture: 92.46%, 97.77%, 99.48%, and 100%. The slides say a normal target is 85% to 95%, so for Iris even 1 component

would pass. In a real dataset with 200 features, like the example on slide 23, I can see how cutting down to 20 components would save a ton of training time.

My guess going in was that if you ran K-Means on my 2 components, it would find setosa easily but mix up some versicolor and virginica, since they overlap in my plot. Thomas ended up testing exactly that in Section 5 using my X_pca output, and that is what happened. Setosa came out perfect, and 14 virginica got grouped in with versicolor. It was cool to see my section get used as the input for someone else's code instead of just being a plot at the end. It also made me realize my output had to be right, because his whole section depended on it.

Reflective Journal

Module 10 Lab: Unsupervised Learning
Group: FA26_ML_6171_16347 5 | Course: ITAI 1371

Thomas Jaymon

Section 5: Extension combining PCA and K-Means, Knowledge Check, and final notebook review

The responsibilities assigned to me in Section 5 were twofold: first, to design and implement an extension that integrated the two principal techniques of the lab, and second, to complete the Knowledge Check and conduct a final review of the notebook. In retrospect, the extension proved to be the more intellectually significant of the two, as it compelled me to examine how clustering and dimensionality reduction function not as isolated procedures but as stages within a single analytical pipeline.

As originally structured, the notebook applied K-Means exclusively to synthetic customer data and PCA exclusively to the Iris dataset. Slide 25, however, describes a combined workflow in which dimensionality is first reduced and clustering is subsequently performed on the resulting principal components. I implemented this workflow by applying K-Means with k = 3 to the two component representation produced in Section 4. The selection of k was grounded in domain knowledge, since the Iris dataset contains three species, rather than in the inertia and silhouette criteria that informed the choice in Section 2. This distinction is worth noting: in applied settings, the number of clusters may be dictated by prior knowledge of the domain as legitimately as by an internal metric.

The results were instructive precisely because they were imperfect. All 50 setosa specimens were assigned to a single cluster without error. Versicolor was largely preserved, with 47 of 50 specimens grouped together, whereas 14 virginica specimens were absorbed into the versicolor cluster. In aggregate, approximately 88.7 percent of specimens were grouped with their own species, yielding an Adjusted Rand Index of 0.716. When contrasted with the perfect score of 1.000 obtained on the synthetic data in Section 3, this outcome illustrates a fundamental property of the algorithm: K-Means can only recover structure that is genuinely present in the feature space. Where two classes overlap, as versicolor and virginica visibly do in the PCA projection, no parameter adjustment can produce a clean separation.

To evaluate the cost of dimensionality reduction, I repeated the clustering on all four original features, which yielded an Adjusted Rand Index of 0.730. The difference of 0.014 is marginal. My prior assumption had been that dimensionality reduction necessarily trades accuracy for computational efficiency. This comparison refined that view considerably: when features are highly correlated, as the Iris measurements are, reducing dimensionality removes redundancy far more than it removes information.

With respect to the Knowledge Check, I elected to defer my responses until all preceding sections were complete, so that each answer could be supported by empirical results rather than

general definitions. The second question, concerning the selection of k, was addressed using the inertia values reported in Section 2: the reduction from 1,919.36 to 362.47 between k = 3 and k = 4, compared with the modest decline to 188.72 at k = 10. The third question drew upon the 97.77 percent explained variance reported in Section 4, supplemented by the findings of my own extension.

The principal difficulty I encountered was one of dependency. My implementation required the X_pca variable from Section 4 and the KMeans import from Section 2, and my written responses required the outputs of every other member. This necessitated a careful reading of all prior cells to ensure that variable names and results were used consistently. The final review likewise involved verifying that each interpretation corresponded to its output and that every attribution label matched the Group Members table.

A methodological question remains unresolved for me. Throughout this lab, a known ground truth was available for validation. In most practical applications of unsupervised learning, no such reference exists. Internal measures such as the silhouette coefficient provide one avenue for evaluation, but I suspect that interpretability to domain experts ultimately serves as the decisive criterion. Exploring how practitioners reconcile these two forms of validation is a direction I intend to pursue.